German University in Cairo
Department of Computer Science
Assoc. Prof. Mervat Abu El-Kheir

Architecture of Massively Scalable Applications, Spring 2026
Lab 5 Manual: Databases II (NoSQL)

Welcome to the second Databases Lab! In Lab 4 you connected Spring Boot to PostgreSQL and built relational models with JPA. In this lab you will work with five additional database technologies — MongoDB (document store), Redis (caching layer), Cassandra (wide-column / time-series), Neo4j (graph database), and Elasticsearch (search engine). Each database solves a problem that SQL handles poorly or slowly. By the end of the session you will have a single Spring Boot application connected to all six databases simultaneously.

1 Project Setup

The starter project contains a Spring Boot skeleton with all five NoSQL Spring Data starters pre-configured. A single Docker Compose file spins up every database you need.

Clone the repo that has the base code to start working on this lab manual through this link `https://github.com/Scalable-2026/Lab05-Base-Code.git`

a) Open a terminal in the project root directory and start all databases:

```
docker compose up -d
```

Verify every container is running:

```
docker ps
```

You should see six containers: `postgres` (port 5432), `mongo` (27017), `redis` (6379), `cassandra` (9042), `neo4j` (7474/7687), and `elasticsearch` (9200). Cassandra takes 30–60 seconds to become healthy — the healthcheck in Compose handles the wait. The Cassandra service automatically creates the `lab05ks` keyspace on first boot.

b) Open `src/main/resources/application.yml` and confirm the connection properties match the Compose file:

```yaml
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/lab05db
    username: postgres
    password: postgres
  jpa:
    hibernate.ddl-auto: update
    show-sql: true

  data:
    mongodb:
      uri: mongodb://root:rootpass@localhost:27017/lab05mongo?authSource=admin

    redis:
      host: localhost
      port: 6379
      password: redispass

    cassandra:
      contact-points: localhost
```

```
    port: 9042
    local-datacenter: datacenter1
    keyspace-name: lab05ks
    schema-action: create-if-not-exists

  neo4j:
    uri: bolt://localhost:7687
    authentication:
      username: neo4j
      password: neo4jpass

elasticsearch:
  uris: http://localhost:9200
```

Each database uses its own Spring Data module. Spring Boot auto-configures separate connection pools, templates, and repositories for each one — no conflicts.

c) Run the application and hit `http://localhost:8080/api/health`. You should receive:

```
OK
```

If any database container is still starting, the app may fail on boot. Wait for all healthchecks to pass and try again.

2 MongoDB — The Document Store

In Lab 4 you stored semi-structured data using JSONB inside PostgreSQL. MongoDB takes that concept further — *every* record is a JSON document, and documents in the same collection can have completely different fields. No schema migrations, no `ALTER TABLE`.

2.1 Defining a Document Entity

a) Open the `MongoProduct` class in the `model.mongo` package and add the document mapping:

```java
@Document(collection = "products")
public class MongoProduct {

    @Id
    private String id;    // MongoDB generates an ObjectId

    @Indexed
    private String name;

    private String category;
    private Double price;
    private Double rating;

    private List<String> tags = new ArrayList<>();

    // Embedded sub-document -- no JOIN needed
    private Map<String, Object> specifications = new HashMap<>();
}
```

Key differences from JPA: `@Document` replaces `@Entity`. The `@Id` field is a `String` (not `Long`) because MongoDB uses ObjectId by default. `@Indexed` creates a MongoDB index on that field for fast lookups. The `specifications` map and `tags` list are stored directly inside the document — arrays and nested objects are first-class citizens, not separate tables.

b) Open `MongoProductRepository` and extend `MongoRepository`:

```
@Repository
public interface MongoProductRepository
        extends MongoRepository<MongoProduct, String> {

    // Pattern 1: Derived queries (auto-generated from method name)
    List<MongoProduct> findByCategory(String category);
    List<MongoProduct> findByPriceBetween(
        Double min, Double max);

    // Pattern 2: @Query with MongoDB JSON syntax
    @Query("{ 'tags': ?0 }")
    List<MongoProduct> findByTag(String tag);
}
```

There are three ways to query MongoDB in Spring Data. **Derived queries** (like `findByCategory`) work exactly like JPA — Spring parses the method name and generates the query. **@Query** uses MongoDB's native JSON syntax instead of JPQL. The third way — **MongoTemplate** — gives you full programmatic control and is what we will use next.

2.2 CRUD and Embedded Documents

a) Open `MongoProductService` and implement the CRUD methods marked with `// TODO` comments. Each method delegates to the repository, following the same pattern as Lab 4.

b) Uncomment the `MongoProductController` and test with Postman. Insert the following three products:

```
POST http://localhost:8080/api/mongo/products
{
  "name": "MacBook Pro",
  "category": "ELECTRONICS",
  "price": 2499.00,
  "rating": 4.8,
  "tags": ["laptop", "apple", "pro"],
  "specifications": {
    "cpu": "M3 Max",
    "ram": "36GB",
    "storage": "1TB SSD"
  }
}
```

```
POST http://localhost:8080/api/mongo/products
{
  "name": "North Face Puffer Jacket",
  "category": "CLOTHING",
  "price": 320.00,
  "rating": 4.5,
  "tags": ["jacket", "winter", "outdoor"],
  "specifications": {
    "size": "L",
    "material": "Nylon",
    "waterproof": true
  }
}
```

```
POST http://localhost:8080/api/mongo/products
{
```

```
  "name": "Designing Data-Intensive Applications",
  "category": "BOOKS",
  "price": 45.99,
  "rating": 4.9,
  "tags": ["databases", "distributed-systems", "backend"],
  "specifications": {
    "author": "Martin Kleppmann",
    "pages": 616,
    "publisher": "O'Reilly"
  }
}
```

Notice that each product has completely different `specifications` fields — the electronics product has `cpu` and `ram`, the clothing product has `size` and `material`, and the book has `author` and `pages`. All in the same collection, no NULLs, no schema migrations.

c) Test the query endpoints:

```
GET /api/mongo/products/category/ELECTRONICS
GET /api/mongo/products/price?min=100&max=3000
GET /api/mongo/products/tag/laptop
```

2.3 MongoTemplate — Aggregation Repository

The `MongoProductRepository` interface handles simple lookups and annotation-based queries, but `MongoRepository` cannot express aggregation pipelines or fully dynamic queries. For those you need `MongoTemplate`. To keep the layered architecture clean (only the repository layer touches the database), the starter project includes a second repository class: `MongoAggregationRepository`. It is a plain `@Repository` class that injects `MongoTemplate` — the service calls it just like any other repository.

a) Open `MongoAggregationRepository` in the `repository.mongo` package. Inject `MongoTemplate` via constructor injection:

```java
// Pattern 3: MongoTemplate -- programmatic queries
@Repository
public class MongoAggregationRepository {

    private final MongoTemplate mongoTemplate;

    public MongoAggregationRepository(
            MongoTemplate mongoTemplate) {
        this.mongoTemplate = mongoTemplate;
    }
}
```

This is a regular class, not an interface — because you need constructor injection and real method bodies. `MongoProductRepository` must be an interface (Spring Data requires it to auto-generate CRUD), but nothing stops you from having a second `@Repository` that uses `MongoTemplate` directly.

b) Implement `getAverageByCategory()` using an Aggregation pipeline. The starter code has the following TODO:

```java
// TODO: Implement getAverageByCategory()
// Use MongoTemplate with an Aggregation pipeline:
// 1. $match -- filter products where rating > 4.0
// 2. $group -- group by category, calculate average price
// 3. $sort  -- sort by avgPrice descending
```

```
// 4. $limit -- take top 5
//
// Return List<CategoryAvgDTO>
```

Aggregation pipelines are MongoDB's most powerful feature — they chain stages like Unix pipes: data flows through `$match` → `$group` → `$sort` → `$limit`.

c) Open `MongoProductService`. Inject both repositories:

```
private final MongoProductRepository repo;
private final MongoAggregationRepository aggregationRepo;

public MongoProductService(
        MongoProductRepository repo,
        MongoAggregationRepository aggregationRepo) {
    this.repo = repo;
    this.aggregationRepo = aggregationRepo;
}
```

The service delegates CRUD to `repo` and aggregation to `aggregationRepo`. The service never touches `MongoTemplate` directly — that stays in the repository layer.

d) Implement `getAverageByCategory()` in the service by delegating to the repository:

```
public List<CategoryAvgDTO> getAverageByCategory() {
    return aggregationRepo.getAverageByCategory();
}
```

e) Expose `GET /api/mongo/products/analytics/category-avg` and verify the results match what you expect from the data you inserted.

3 Redis — The In-Memory Speed Layer

Redis stores all data in RAM. A PostgreSQL query takes 5–50 ms. A Redis lookup takes 0.1 ms. You do not replace your database with Redis — you put Redis *in front of it* as a cache. When 1,000 users request the same product page, only the first request hits the database; the other 999 are served from Redis in sub-millisecond time.

3.1 Cache Configuration

a) Open `RedisConfig` in the `config` package. The starter code has a TODO to create the `RedisCacheManager` bean:

```
@Configuration
@EnableCaching
public class RedisConfig {

    @Bean
    public RedisCacheManager cacheManager(
            RedisConnectionFactory factory) {

        // JSON serialization so cached data is human-readable
        RedisSerializationContext.SerializationPair<Object>
            jsonSerializer = RedisSerializationContext
                .SerializationPair.fromSerializer(
                    new GenericJackson2JsonRedisSerializer());
```

```
        RedisCacheConfiguration config =
            RedisCacheConfiguration.defaultCacheConfig()
                .serializeValuesWith(jsonSerializer)
                .entryTtl(Duration.ofMinutes(30));
                // TTL = 30 min -- cached entries expire automatically

        return RedisCacheManager.builder(factory)
                .cacheDefaults(config)
                .build();
    }
}
```

Without `@EnableCaching`, all `@Cacheable` annotations are **silently ignored** — this is the number-one Redis debugging headache. The TTL (time-to-live) ensures stale data is eventually evicted. A sticky note that never expires will eventually lie to you.

3.2 Caching Product Lookups

a) Open the `ProductService` that works with the PostgreSQL `Product` entity from Lab 4. Add caching annotations:

```
@Cacheable(value = "products", key = "#id")
public Product getProductById(Long id) {
    log.info("CACHE MISS -- hitting database for product {}", id);
    return productRepository.findById(id)
        .orElseThrow(() -> new RuntimeException(
            "Product not found: " + id));
}

@CachePut(value = "products", key = "#product.id")
public Product updateProduct(Product product) {
    log.info("Updating product {} and refreshing cache", product.getId());
    return productRepository.save(product);
}

@CacheEvict(value = "products", key = "#id")
public void deleteProduct(Long id) {
    log.info("Deleting product {} and evicting from cache", id);
    productRepository.deleteById(id);
}
```

There are four caching annotations to know:

- `@Cacheable` — read-through: return from cache on HIT, call method + store result on MISS. Use for GET operations (90% of caching).
- `@CachePut` — write-through: always execute the method AND update the cache. Use for UPDATE operations.
- `@CacheEvict` — invalidate: remove the entry from cache. Use for DELETE operations.
- `@Caching` — compose multiple cache operations on one method.

b) Test the caching behavior. First, create a product in PostgreSQL:

```
POST http://localhost:8080/api/products
{
  "name": "Wireless Mouse",
  "category": "ELECTRONICS",
  "price": 29.99,
  "stockQuantity": 150
}
```

Now test caching on the GET endpoint:

```
# First call -- check the console for "CACHE MISS"
GET http://localhost:8080/api/products/1

# Second call -- no "CACHE MISS" log, response is instant
GET http://localhost:8080/api/products/1
```

The first request takes the full database round-trip. The second request returns the cached value without executing the method body at all.

c) Verify eviction works: call `DELETE /api/products/1`, then `GET /api/products/1` — you should see "CACHE MISS" again because the entry was evicted.

4 Cassandra — Built for Massive Writes

Cassandra is a distributed wide-column store built for *write-heavy* workloads at enormous scale. Netflix logs billions of play events per day. Discord stores millions of messages per second. The key insight is that Cassandra distributes data across many nodes using a **Partition Key**, and sorts rows within each partition using a **Clustering Column**.

4.1 The Compound Primary Key

a) Open `SensorReadingKey` in the `model.cassandra` package and define the compound primary key:

```java
@PrimaryKeyClass
public class SensorReadingKey implements Serializable {

    @PrimaryKeyColumn(name = "sensor_id",
        ordinal = 0,
        type = PrimaryKeyType.PARTITIONED)
    private String sensorId;

    @PrimaryKeyColumn(name = "reading_time",
        ordinal = 1,
        type = PrimaryKeyType.CLUSTERED,
        ordering = Ordering.DESCENDING)
    private Instant readingTime;
}
```

The partition key (`PARTITIONED`) determines *which node* stores the data. All rows with `sensor_id = "sensor-42"` live on the same node. The clustering column (`CLUSTERED`) determines the *sort order* within that partition. `DESCENDING` means the newest reading comes first — range queries on time are free because data is physically sorted on disk.

b) Open the `SensorReading` entity:

```java
@Table("sensor_readings")
public class SensorReading {

    @PrimaryKey
    private SensorReadingKey key;

    private Double temperature;
    private Double humidity;
    private String location;
}
```

c) Open `SensorReadingRepository`:

```
@Repository
public interface SensorReadingRepository
        extends CassandraRepository<SensorReading,
                                    SensorReadingKey> {

    // Pattern 1: Derived query (auto-generated)
    List<SensorReading> findByKeySensorId(String sensorId);

    // Pattern 2: @Query with CQL
    @Query("SELECT * FROM sensor_readings " +
           "WHERE sensor_id = ?0 " +
           "AND reading_time >= ?1 AND reading_time <= ?2")
    List<SensorReading> findReadingsInRange(
        String sensorId, Instant from, Instant to);
}
```

The golden rule: every Cassandra query **must** include the partition key in the WHERE clause. A query without the partition key scans the entire cluster and will time out on large datasets — this is why `ALLOW FILTERING` is a red flag.

4.2 CassandraTemplate — Programmatic Queries

Just like MongoDB has `MongoTemplate`, Cassandra has `CassandraTemplate` for queries that go beyond derived methods and `@Query` annotations. To keep the layered architecture clean, these queries live in a separate `@Repository` class.

a) Open `CassandraQueryRepository` in the `repository.cassandra` package. Inject `CassandraTemplate` via constructor injection and implement the following method:

```
// Pattern 3: CassandraTemplate -- programmatic query
@Repository
public class CassandraQueryRepository {

    private final CassandraTemplate cassandraTemplate;

    public CassandraQueryRepository(
            CassandraTemplate cassandraTemplate) {
        this.cassandraTemplate = cassandraTemplate;
    }

    public List<SensorReading> findLatestReadings(
            String sensorId, int limit) {
        // TODO: Build a CQL query using:
        // Query.query(Criteria.where("sensor_id")
        //         .is(sensorId))
        //     .limit(limit);
        // Execute with cassandraTemplate.select(query,
        //                        SensorReading.class);
    }
}
```

`CassandraTemplate` is useful when the limit or filter criteria are dynamic at runtime. The derived method `findByKeySensorId` always returns *all* readings; here you can cap the result programmatically.

4.3 Writing and Reading Sensor Data

a) Open `SensorService`. Inject both `SensorReadingRepository` and `CassandraQueryRepository`. Implement the TODO methods — the service delegates to the interface for derived and `@Query` methods, and to the class for template-based queries.

b) Uncomment the `SensorController` and test:

```
POST http://localhost:8080/api/sensors/readings
{
  "sensorId": "sensor-42",
  "temperature": 23.5,
  "humidity": 65.2,
  "location": "Building A - Floor 3"
}
```

Insert five or six readings for the same sensor at different times. Then query them:

```
GET /api/sensors/sensor-42/readings
GET /api/sensors/sensor-42/readings/latest?limit=3
```

Notice the results are already sorted by `reading_time` descending — newest first, with zero sorting cost at query time.

5 Neo4j — When the Relationship IS the Data

In PostgreSQL, relationships are *computed* at query time via JOINs. In Neo4j, relationships are *stored physically* as pointers between nodes — traversal is instant regardless of dataset size. Finding "friends of friends of friends" takes 5 ms in Neo4j whether you have 1,000 or 1,000,000,000 users. In SQL the same query degrades exponentially with depth.

5.1 Defining Nodes and Relationships

a) Open the `Person` class in the `model.neo4j` package:

```java
@Node
public class Person {

    @Id @GeneratedValue
    private Long id;

    private String name;

    @Relationship(type = "FOLLOWS",
                  direction = Relationship.Direction.OUTGOING)
    private List<Person> following = new ArrayList<>();

    @Relationship(type = "PURCHASED",
                  direction = Relationship.Direction.OUTGOING)
    private List<Purchased> purchases = new ArrayList<>();
}
```

`@Node` replaces `@Entity`. Relationships are typed and directed — `Alice -[:FOLLOWS]-> Bob` does not imply Bob follows Alice. If you know JPA, you already know 80% of Spring Data Neo4j: replace `@Entity` → `@Node`, `JpaRepository` → `Neo4jRepository`, SQL → Cypher.

b) Open the `Purchased` relationship class:

```java
@RelationshipProperties
public class Purchased {

    @Id @GeneratedValue
    private Long id;

    @TargetNode
    private Neo4jProduct product;

    private LocalDateTime purchasedAt;
    private Integer quantity;
}
```

In JPA, a foreign key cannot carry data — you need a join entity. In Neo4j, the relationship *itself* has properties. "Alice PURCHASED MacBook" also stores when, how many, and at what price — all on the edge.

c) Open `PersonRepository` with custom Cypher queries:

```java
@Repository
public interface PersonRepository
        extends Neo4jRepository<Person, Long> {

    // Pattern 1: Derived query (auto-generated)
    Optional<Person> findByName(String name);

    // Pattern 2: @Query with Cypher
    // Friends of friends (2nd-degree connections)
    @Query("MATCH (p:Person {name: $name})" +
           "-[:FOLLOWS*2]-(fof:Person) " +
           "WHERE fof <> p " +
           "AND NOT (p)-[:FOLLOWS]-(fof) " +
           "RETURN DISTINCT fof")
    List<Person> findFriendsOfFriends(
        @Param("name") String name);
}
```

Cypher reads like ASCII art of the graph pattern you want to find. `(p)-[:FOLLOWS*2]-(fof)` means "traverse exactly 2 FOLLOWS hops from p." The third query pattern — **Neo4jClient** — gives you full programmatic control for complex queries that return custom projections (not full nodes).

5.2 Neo4jClient — Programmatic Graph Queries

`Neo4jRepository` `@Query` methods work well when you return full `@Node` entities, but the collaborative filtering recommendation query returns aggregated data (product names and friend counts) — not a `Person` or `Neo4jProduct` node. For this, `Neo4jClient` gives you full control over the Cypher query and result mapping. To maintain the layered architecture, this lives in its own `@Repository` class.

a) Open `Neo4jGraphRepository` in the `repository.neo4j` package. Inject `Neo4jClient` and implement the recommendation query:

```java
// Pattern 3: Neo4jClient -- programmatic Cypher
@Repository
public class Neo4jGraphRepository {

    private final Neo4jClient neo4jClient;

    public Neo4jGraphRepository(Neo4jClient neo4jClient) {
```

```java
        this.neo4jClient = neo4jClient;
    }

    public List<Map<String, Object>> getRecommendations(
            String personName, int limit) {
        return neo4jClient
            .query("MATCH (p:Person {name: $name})" +
                   "-[:FOLLOWS]->(friend)" +
                   "-[:PURCHASED]->(prod) " +
                   "WHERE NOT (p)-[:PURCHASED]->(prod) " +
                   "RETURN prod.name AS product, " +
                   "       COUNT(friend) AS score " +
                   "ORDER BY score DESC LIMIT $limit")
            .bind(personName).to("name")
            .bind(limit).to("limit")
            .fetch()
            .all()
            .stream().toList();
    }
}
```

Understanding the Cypher query — read it like a sentence: "Start at person `$name`, follow FOLLOWS edges to their friends, then follow PURCHASED edges to products those friends bought. Exclude any product that `$name` already purchased. Count how many friends bought each product, and return the top results."

For example, given this graph:

```
Alice --FOLLOWS--> Bob --PURCHASED--> MacBook
Alice --FOLLOWS--> Carol --PURCHASED--> MacBook
Alice --FOLLOWS--> Carol --PURCHASED--> AirPods
Alice --PURCHASED--> iPhone (Alice already owns this)
```

The result for Alice would be:

```
{ "product": "MacBook", "score": 2 } // Bob + Carol
{ "product": "AirPods", "score": 1 } // only Carol
```

iPhone is excluded because Alice already purchased it — that is the `WHERE NOT` clause.

Why `List<Map<String, Object>>` and not a `@Node` entity? The query does not return a full node — it returns `prod.name` (a String) and `COUNT(friend)` (a number). That is a custom projection. `Neo4jClient` maps each result row to a `Map` where the keys (`"product"`, `"score"`) come from the `AS` aliases in the Cypher `RETURN` clause.

How the binding works:

```
.bind(personName).to("name") // Java variable -> Cypher $name
.bind(limit).to("limit")     // Java variable -> Cypher $limit
```

Wherever Cypher sees `$name`, it is replaced with the value of the Java variable `personName`. This is the same concept as `@Param("name")` in repository `@Query` methods, just with a fluent builder syntax. `.fetch().all()` executes the query and collects every row into a collection of maps.

5.3 Building the Social Graph

a) Open `SocialGraphService`. Inject both `PersonRepository` (for CRUD and `@Query` methods) and `Neo4jGraphRepository` (for the recommendation query). Implement the TODO methods for creating persons, following, and purchasing.

b) Uncomment the `SocialGraphController` and test:

```
# Create people
POST /api/graph/persons {"name": "Alice"}
POST /api/graph/persons {"name": "Bob"}
POST /api/graph/persons {"name": "Carol"}

# Alice follows Bob, Bob follows Carol
POST /api/graph/persons/Alice/follow/Bob
POST /api/graph/persons/Bob/follow/Carol

# Bob purchases a product
POST /api/graph/persons/Bob/purchase
{"productName": "MacBook", "quantity": 1}

# Alice's recommendations (products her friends bought)
GET /api/graph/persons/Alice/recommendations?limit=5
```

c) Open `http://localhost:7474` in a browser to access the Neo4j Browser. Run the Cypher query `MATCH (n) RETURN n` to see a visual graph of all nodes and relationships you created. This visual explorer is invaluable for debugging graph queries.

6 Elasticsearch — When Search IS the Feature

Try finding products matching "wirelss headpones" (with typos!) in PostgreSQL using `LIKE`. On 10 million products it takes 4–8 seconds, has zero typo tolerance, and no relevance ranking. Elasticsearch handles the same query in 5 ms with built-in fuzziness and TF-IDF/BM25 relevance scoring.

Elasticsearch is **not** a primary data store. It is a search *index*. Your database (PostgreSQL or MongoDB) is the source of truth. You sync data *to* Elasticsearch to make it searchable.

6.1 Defining a Searchable Document

a) Open `ProductDocument` in the `model.elastic` package:

```java
@Document(indexName = "products")
public class ProductDocument {

    @Id
    private String id;

    @Field(type = FieldType.Text, analyzer = "english")
    private String name;

    @Field(type = FieldType.Text, analyzer = "english")
    private String description;

    @Field(type = FieldType.Keyword)
    private String category;

    @Field(type = FieldType.Double)
    private Double price;

    @Field(type = FieldType.Keyword)
    private List<String> tags;

    @Field(type = FieldType.Boolean)
    private Boolean inStock;
}
```

The field type is critical in Elasticsearch. `Text` fields are tokenized, lowercased, and stemmed — searching "headphone" finds "headphones." `Keyword` fields are stored as-is for exact matching, sorting, and aggregation — searching "electronic" does **not** find "electronics." Rule: if users *search* for it (name, description) → `Text`. If users *filter* by it (category, status) → `Keyword`.

b) Open `ProductSearchRepository`:

```
@Repository
public interface ProductSearchRepository
        extends ElasticsearchRepository<ProductDocument,
                                        String> {

    // Pattern 1: Derived query (auto-generated)
    List<ProductDocument> findByCategory(String category);
    List<ProductDocument> findByPriceBetween(
        Double min, Double max);

    // Pattern 2: @Query with Elasticsearch JSON
    @Query("{\"match\": {\"name\": " +
           "{\"query\": \"?0\", \"fuzziness\": \"AUTO\"}}}")
    List<ProductDocument> searchByName(String name);
}
```

Derived queries work on `Keyword` and numeric fields, but **not** on `Text` fields (because `Text` is tokenized). The `@Query` annotation uses Elasticsearch's native JSON Query DSL — here it performs a fuzzy `match` on the `name` field. For more complex queries (multi-field search, bool queries, boosting), use `ElasticsearchOperations`.

6.2 ElasticsearchOperations — Full-Text Search

For production-grade search (multi-field matching with typo tolerance, field boosting, filters, and pagination), you need the full `NativeQuery` builder. To maintain the layered architecture, this logic lives in its own `@Repository` class.

a) Open `ElasticSearchQueryRepository` in the `repository.elastic` package. Inject `ElasticsearchOperations` and implement the search method. The starter code has the following TODO:

```
// Pattern 3: ElasticsearchOperations -- programmatic queries
@Repository
public class ElasticSearchQueryRepository {

    private final ElasticsearchOperations esOps;

    public ElasticSearchQueryRepository(
           ElasticsearchOperations esOps) {
        this.esOps = esOps;
    }

    // TODO: Implement search(String query, String category,
    //                        Double minPrice, Double maxPrice,
    //                        int page, int size)
    //
    // Use NativeQuery with a Bool query:
    // 1. must: multi_match on "name" and "description"
    //          with fuzziness("AUTO") and name boosted (^3)
    // 2. filter: term on "category" (if provided)
    // 3. filter: range on "price" (if min/max provided)
    // 4. Paginate with PageRequest.of(page, size)
```

```
    // 5. Execute with esOps.search(...)
    // 6. Map SearchHits to List<ProductDocument>
}
```

The key distinction: `must` makes matching documents rank higher (a match in the product name scores 3× higher than a match in the description). `filter` is a binary gate — the document either passes or it does not, with no impact on relevance score. Category and price filters belong in `filter`, not `must`.

b) Open `ProductSearchService`. Inject both `ProductSearchRepository` (for CRUD, derived and `@Query` methods) and `ElasticSearchQueryRepository` (for the full-text search). The service delegates to each without touching `ElasticsearchOperations` directly.

c) Uncomment the `SearchController` and test. To populate the index, insert the following products:

```
POST http://localhost:8080/api/search/products
{
  "id": "1",
  "name": "Wireless Headphones",
  "description": "Premium noise-cancelling bluetooth headphones with 30hr battery",
  "category": "ELECTRONICS",
  "price": 199.99,
  "tags": ["audio", "wireless", "bluetooth"],
  "inStock": true
}
```

```
POST http://localhost:8080/api/search/products
{
  "id": "2",
  "name": "Wireless Earbuds",
  "description": "Compact in-ear wireless earbuds with active noise cancellation",
  "category": "ELECTRONICS",
  "price": 129.99,
  "tags": ["audio", "wireless", "earbuds"],
  "inStock": true
}
```

```
POST http://localhost:8080/api/search/products
{
  "id": "3",
  "name": "Running Shoes",
  "description": "Lightweight mesh running shoes for marathon training",
  "category": "SPORTS",
  "price": 89.99,
  "tags": ["running", "fitness", "shoes"],
  "inStock": true
}
```

```
POST http://localhost:8080/api/search/products
{
  "id": "4",
  "name": "Mechanical Keyboard",
  "description": "RGB mechanical gaming keyboard with Cherry MX switches",
  "category": "ELECTRONICS",
  "price": 149.99,
  "tags": ["keyboard", "gaming", "mechanical"],
  "inStock": false
}
```

Now test the three query patterns:

```
# Pattern 1: Derived query (Keyword field -- exact match)
# Returns: Wireless Headphones, Wireless Earbuds, Mechanical Keyboard
GET /api/search/products/category/ELECTRONICS

# Pattern 2: @Query with fuzzy match (tolerates typos)
# "headpones" matches "Headphones" (1 edit: insert h)
# Returns: Wireless Headphones
GET /api/search/products/name?q=headpones

# Pattern 3: Full NativeQuery search (multi-field + filters)
# "wirelss" matches "Wireless" (1 edit: insert e)
# Filtered to ELECTRONICS between $50-$500
# Returns: Wireless Headphones (score higher -- name match)
#         Wireless Earbuds (score lower)
GET /api/search?q=wirelss+headpones&category=ELECTRONICS
             &minPrice=50&maxPrice=500
```

The `fuzziness("AUTO")` parameter allows 1–2 character edits: "wirelss" → "wireless", "headpones" → "headphones." Notice that Pattern 3 returns both wireless products but ranks Headphones higher because "headpones" matches the name field (boosted 3×). The Running Shoes are excluded by the category filter, and the Mechanical Keyboard is included because it matches ELECTRONICS even though it does not match the search query (it would score lower).

7 Polyglot Integration — All Databases Working Together

Each database you used in this lab excels at one thing:

Database	Strength	Example Use
PostgreSQL	ACID transactions	Orders, payments, money
MongoDB	Flexible schema	Product catalogs, CMS
Redis	Sub-ms speed	Caching, sessions, leaderboards
Cassandra	Massive write throughput	Event streams, time-series logs
Neo4j	Relationship traversal	Social graphs, recommendations
Elasticsearch	Full-text search	Product search, autocomplete

The starter project includes a `SmartProductPageService` that demonstrates how a single API call can touch multiple databases. Imagine a user opens a product page on an e-commerce site. That single page needs to show:

- The product details — name, images, flexible specs → **MongoDB** (flexible schema per product type)
- The real-time price and stock count — must be accurate → **PostgreSQL** (ACID, source of truth)
- "Related products you might like" — text similarity → **Elasticsearch** (relevance-scored search)
- "3 of your friends bought this" — social proof → **Neo4j** (graph traversal)
- The page loads in under 50 ms because it was cached → **Redis** (sub-ms response)
- The fact that you viewed this page is logged for analytics → **Cassandra** (append-only event stream)

One API call, one response, six databases — each contributing the one thing it does best. That is polyglot persistence in action.

7.1 The SmartProductPage DTO

a) In the `dto` package, create the response record that assembles data from all six databases:

```
public record SmartProductPage(
    String name,                    // PostgreSQL
    Double price,                   // PostgreSQL
    Integer stock,                  // PostgreSQL
    Map<String, Object> specifications, // MongoDB
    List<String> tags,              // MongoDB
    List<String> relatedProducts,   // Elasticsearch
    List<String> friendsWhoBought,  // Neo4j
    boolean servedFromCache         // Redis
) implements Serializable {}
```

Every field comes from a different database. This is polyglot persistence at its purest. Because records are immutable (no setters), the service will collect data into local variables and construct the record at the end.

7.2 The SmartProductPageService

a) Open `SmartProductPageService`. Inject the services from each database:

```
@Service
public class SmartProductPageService {

    private final ProductService pgService;
    private final MongoProductService mongoService;
    private final ProductSearchService esService;
    private final SocialGraphService graphService;
    private final SensorService cassandraService;
    private final RedisTemplate<String, SmartProductPage> redisTemplate;

    // Constructor injection for all six
}
```

b) Implement the `getSmartPage` method. The key design pattern is **graceful degradation** — only PostgreSQL is a hard dependency; every other database is wrapped in `try-catch` so the page still loads if any non-critical service is down. Because `SmartProductPage` is a record (immutable — no setters), you collect data into local variables and construct the record at the end:

```
public SmartProductPage getSmartPage(Long productId,
                                     String userName) {

    // 0. Redis -- CHECK CACHE FIRST
    String cacheKey = "smart-page:" + productId
                    + ":" + userName;
    SmartProductPage cached = redisTemplate
        .opsForValue().get(cacheKey);
    if (cached != null) {
        // Record is immutable -- reconstruct with flag
        return new SmartProductPage(
            cached.name(), cached.price(), cached.stock(),
            cached.specifications(), cached.tags(),
            cached.relatedProducts(),
            cached.friendsWhoBought(), true);
    }

    // Collect into local variables (records have no setters)
```

```java
    // 1. PostgreSQL -- source of truth (HARD dependency)
    Product product = pgService.getProductById(productId);
    String name = product.getName();
    Double price = product.getPrice();
    Integer stock = product.getStockQuantity();

    // 2. MongoDB -- flexible specs (soft dependency)
    Map<String, Object> specifications = null;
    List<String> tags = null;
    try {
        List<MongoProduct> mongoResults =
            mongoService.getByCategory(
                product.getCategory());
        if (!mongoResults.isEmpty()) {
            MongoProduct mp = mongoResults.get(0);
            specifications = mp.getSpecifications();
            tags = mp.getTags();
        }
    } catch (Exception e) { /* MongoDB down */ }

    // 3. Elasticsearch -- related products (soft)
    List<String> relatedProducts = null;
    try {
        List<ProductDocument> related =
            esService.searchByName(product.getName());
        relatedProducts = related.stream()
            .map(ProductDocument::getName)
            .limit(3).toList();
    } catch (Exception e) { /* ES down */ }

    // 4. Neo4j -- friends who bought (soft)
    List<String> friendsWhoBought = null;
    try {
        List<Map<String, Object>> recs =
            graphService.getRecommendations(userName, 3);
        friendsWhoBought = recs.stream()
            .map(r -> (String) r.get("product"))
            .toList();
    } catch (Exception e) { /* Neo4j down */ }

    // 5. Cassandra -- log the view event (fire & forget)
    try {
        // ... record the event ...
    } catch (Exception e) { /* Cassandra down */ }

    // 6. Construct the record and cache it
    SmartProductPage page = new SmartProductPage(
        name, price, stock,
        specifications, tags,
        relatedProducts, friendsWhoBought,
        false // freshly assembled
    );

    redisTemplate.opsForValue()
        .set(cacheKey, page, Duration.ofMinutes(2));

    return page;
}
```

Notice the pattern: step 0 checks Redis *before* touching any database — if the page was assembled recently, it reconstructs the record with `servedFromCache = true` (records are immutable, so you

cannot call a setter — you create a new instance). Step 6 constructs the record from all the local variables and caches it. The `servedFromCache` field lets you verify in the response whether the data came from cache or was freshly assembled.

7.3 Controller and Testing

a) Create `SmartProductPageController`:

```
@RestController
@RequestMapping("/api/smart")
public class SmartProductPageController {

    private final SmartProductPageService service;

    public SmartProductPageController(
            SmartProductPageService service) {
        this.service = service;
    }

    @GetMapping("/product/{id}")
    public SmartProductPage getPage(
            @PathVariable Long id,
            @RequestParam(defaultValue = "Alice")
                String userName) {
        return service.getSmartPage(id, userName);
    }
}
```

b) Make sure you have data in all databases from the previous sections. You should have:

- A PostgreSQL product (created in Section 3 — the "Wireless Mouse")
- MongoDB products (created in Section 2 — MacBook Pro, Jacket, Book)
- Elasticsearch products (created in Section 6 — Wireless Headphones, etc.)
- Neo4j persons with purchases (created in Section 5 — Alice, Bob, Carol)

c) Test the smart product page:

```
GET http://localhost:8080/api/smart/product/1?userName=Alice
```

The response assembles data from all six databases into a single JSON object:

```
{
  "name": "Wireless Mouse",
  "price": 29.99,
  "stock": 150,
  "specifications": { "cpu": "M3 Max", ... },
  "relatedProducts": ["Wireless Headphones", ...],
  "friendsWhoBought": ["MacBook"],
  "servedFromCache": false
}
```

Each field was fetched from a different database. Notice `"servedFromCache": false` — this response was freshly assembled from all six databases.

d) Call the same endpoint again immediately:

```
GET http://localhost:8080/api/smart/product/1?userName=Alice
```

This time the response returns instantly and shows:

```
{
  ...
  "servedFromCache": true
}
```

The second call skipped all five databases entirely — Redis returned the cached page from step 0. The cache entry expires after 2 minutes (the TTL set in step 6), after which the next request will rebuild the page from scratch.

e) Test graceful degradation: stop one of the non-critical containers:

```
docker stop lab05-neo4j
```

Wait for the cache to expire (2 minutes), then call the endpoint again. The response still works — the `friendsWhoBought` field will simply be `null` instead of crashing the whole page. Start it back with `docker start lab05-neo4j`.

f) **Important takeaway:** start with PostgreSQL for everything. Extract to a specialized NoSQL database only when you hit a specific, measurable bottleneck. Every new database adds a connection pool, a failure mode, and a schema to maintain. Each one must justify its operational cost.

8 Database Selection Quick Reference

Use this decision guide when choosing a database for a feature:

a) Does it involve money or critical state? → **PostgreSQL** (ACID)

b) Do you need sub-millisecond response? → **Redis** (in-memory)

c) Is full-text search or fuzzy matching needed? → **Elasticsearch** (inverted index)

d) Are relationships the primary query pattern? → **Neo4j** (graph traversal)

e) Do you write millions of events per second? → **Cassandra** (append-only)

f) Does your data schema vary by type? → **MongoDB** (flexible docs)

g) Not sure yet? → **PostgreSQL.** Always start here.

9 Lab Solution

You can clone this repo `https://github.com/Scalable-2026/Lab-05-Solution.git` to access the solution of the lab. You will also find a Postman collection that you can import in Postman and try the different routes.